2-1 路径转换器

需求场景

我们知道路径中的参数（或者说从URL提取出来来的参数）都是字符串类型

需求1：如果我们定义了动态路径参数，那如果需要做类型转化的时候，除了在路径函数内使用类型提示的方式，还有其他方式吗？

需求2：如果我的动态路径参数就是路径，比如：`/files/data/abc.txt`，如何获取`data/abc.txt`这个参数？

FastAPI的解决方式

示例1：使用路径转化器 int，可以直接帮我们将这个参数转化为int类型

```
import typing
from fastapi import FastAPI


app = FastAPI(title="路径转化器")

books = {
    1: {"id": 1, "name": "图书1"},
    2: {"id": 2, "name": "图书2"},
    3: {"id": 3, "name": "图书3"},
}


@app.get("/books/{id:int}")     # 注意 {id:int}  :两边不能有空格
def get_books_by_id(id):
    print(id, type(id))
    return books.get(id)
```

注意1：`get_books_by_id(id)`因为此时没有在路径函数内使用类型提示，所以openapi文档没有了类型校验功能。

注意2：如果参数类型不匹配，则直接返回404 Not Found，程序是不会执行路径函数内的代码。

示例2：普通路径参数无法获取路径值

```python
# main.py
from fastapi import FastAPI

app = FastAPI()


@app.get("/files/{file_path}")
def read_file(file_path: str):
    print(file_path)
    with open(file_path, "r") as f:
        return f.read()
```

- 比如，在当前路径下有一个aaa文件夹，其中有一个abc.txt文件
- 此时，你想通过访问URL：`/files/aaa/abc.txt`，将变量：`aaa/abc.txt`传递给file_path这个形参
- 那这种情况下示例2的代码不能解决我们的需求。
- 此时需要使用路径转化器：path

示例3：使用路径转化器 path，帮我们找到路径值

```python
# main.py
from fastapi import FastAPI

app = FastAPI()


@app.get("/files/{file_path:path}")
def read_file(file_path: str):
    print(file_path)
    with open(file_path, "r") as f:
        return f.read()
```

内置5中路径转化器

- `str` 返回字符串（默认方式）
- `int` 返回Python中的int
- `float` 返回Python中的float.
- `uuid` 返回Python中的`uuid.UUID`
- `path` 返回路径，可以包含任意多个的`/`

> 补充1：这些内置转化器是Fastapi直接复用Starlette中的路径转化器
>
> 补充2：除了这5个转化器，我们也可以自定义符合自己需求的转化器（参考官网代码）
>
> ```python
> from datetime import datetime
>
> from starlette.convertors import Convertor, register_url_convertor
>
>
> class DateTimeConvertor(Convertor):
> regex = "[0-9]{4}-[0-9]{2}-[0-9]{2}T[0-9]{2}:[0-9]{2}:[0-9]{2}(.[0-9]+)?"
>
> def convert(self, value: str) -> datetime:
> ```

```
    return datetime.strptime(value, "%Y-%m-%dT%H:%M:%S")

def to_string(self, value: datetime) -> str:
    return value.strftime("%Y-%m-%dT%H:%M:%S")

register_url_convertor("datetime", DateTimeConvertor())
```

2-2 路径参数枚举值

需求场景

我们有一个博客网站，需要定义一个根据标签来查询博客的接口，比如：`/blogs/{tag}`。

这种情况下，一般有固定的几种标签类型，比如：`tag`的值只有三种类型：python、linux、web

此时在FastAPI中该如何编写这种需求的接口呢？

FastAPI的解决方式

- 因为tag的值只有三个，所以我们可以使用枚举的方式，先定义一个枚举类 `TagName`
- 然后正常编写路径装饰器和路径函数，只不过此时路径参数 `tag`的类型为 `TagName`

```
from enum import Enum

from fastapi import FastAPI


class TagName(str, Enum):
    PYTHON = "python"
    LINUX = "linux"
    WEB = "web"


app = FastAPI()


@app.get("/blogs/{tag}")
def get_blogs_by_tag(tag: TagName):
    if tag == TagName.PYTHON:
        return "some blogs about python"

    if tag.value == "web":
        return "some blogs about web"

    return "some blogs about linux"
```

- `TagName`继承 `Enum`是为了使用枚举类型，继承 `str`是为了有类型提示和API文档显示功能。（python3.11可以直接使用StrEnum）
- 有了枚举类型，可以直接使用枚举变量的 `namne`和 `value`两个属性。

    - name：表示枚举变量的名字，比如， PYTHON
    - value：表示枚举变量的值，比如，python
- 有了`TagName`，FastAPI就知道tag这个路劲参数只有三个字符串类型的值，如果URL中有其他值就会直接报错。

2-3 路径参数使用Path校验

需求场景

我们定义的路径参数如果是字符串，是否可以限制字符串的长度，比如，不能少于几个字符，不能多于几个字符？

我们定义的路径参数如果是数字，是否可以限制数字的大小，比如，不能小于多少，不能大于多少？

FastAPI的解决方式

- 对于上述的需求，FastAPI肯定是可以解决的，我们只要从fastapi中引入`Path`这个函数

示例1：校验路径参数的字符最少3，且最多不能超多10个

```
from fastapi import FastAPI, Path


app = FastAPI()


@app.get("/blog/{name}")
def get_blog_by_name(name: str = Path(min_length=3, max_length=10)):
    return f"the length of this blog name is: {len(name)}"
```

> 补充1：此时name的类型是str是必须的，因为我们使用了字符串校验参数 min_length和max_length
>
> 补充2：只要在路径函数中定义的参数值是Path()，那这个参数必定的路径参数
>
> 补充3：Path是一个函数，会返回一个同名类的实例对象

实例2：校验路径参数的数字不能小于3，不能大于10

```
from fastapi import FastAPI, Path


app = FastAPI()


@app.get("/blog/{id}")
def get_blog_by_id(id: int = Path(ge=3, le=10)):
    return id
```

> 补充：此时id的类型是int这也是必须的，因为我们使用了ge、le，类似的还有：gt、lt

总结

- 只要路径参数通过Path()函数校验，使用了min_length或max_length，则该参数类型必须是str
- 只要路径参数通过Path()函数校验，使用了ge或le等，则该参数类型必须是int
- 如果路径参数使用了Path但并没有具体的校验逻辑，则可以使用类型提示，也可以不使用

2-4 Path之接口文档设置

Path()不仅可以用来做路径参数的校验逻辑，还可以用来设置接口文档的展示功能，常用的设置字段如下：

```
alias:              # 设置别名，修改是要配合路径装饰器中的参数名一块修改
title:              # 设置标题，在redoc中有显示
description:         # 设置参数描述信息
example:            # 设置例子
examples:           # 设置多个例子，比较复杂
deprecated:         # 将要废弃该接口
include_in_schema:  # 是否把schema显示在api文档
```

示例1： examples

```
from fastapi import FastAPI, Path


app = FastAPI()

examples = {
    "valid": {"value": 20},
    "invalid": {"value": 8},
}


@app.get("/blog/{id}")
def get_blog_by_id(id: int = Path(ge=10, examples=examples)):
    return id
```

2-5 查询参数设置默认值

需求场景

比如，图书列表页面，使用page和size两个查询参数做分页，但存在URL中没有携带这俩参数的情况，此时能否有默认值呢？

比如，我们要求前端在调后端接口时，有些查询参数必须要有(必选)，有些可以没有(可选)，该如何实现？

还比如，有的查询参数的类型是 `布尔` ，那在URL中这个参数的值该如何表示呢？

FastAPI的解决方式

示例1：查询参数设置默认值

```
from fastapi import FastAPI

app = FastAPI()

books = [{"id": i, "name": f"图书{i}"} for i in range(10)]


@app.get("/books")
def get_books(page: int = 1, size: int = 5):
    return books[(page - 1) * size:page * size]
```

补充：查询参数使用默认值，可以不使类型提示；使用类型提示的目的：类型校验/类型转换/自动补全，如果没有则需要自己手动转换

示例2：设置查询参数是必选的，可选的

```
import typing
from fastapi import FastAPI

app = FastAPI()

books = [{"id": i, "name": f"图书{i}"} for i in range(10)]


@app.get("/books")
def get_books(page: int, size=3, q: typing.Optional[str] = None):
    size = int(size)
    if q:
        return books
    return books[(page - 1) * size:page * size]
```

结论：

- 只要查询参数没有默认值，则是必填的；只要查询参数的默认值是None，则是选填的
- 如果查询参数有不是None的默认值，则该参数是必填的，但以为有了默认值，所以在URL中可以没有
- 不管有没有默认值，查询参数的类型提示都是用来做类型转换/校验/自动补全的

示例3：查询参数的类型是布尔值

```
import random
```

```python
from fastapi import FastAPI

app = FastAPI()

books = [{"id": i, "name": f"图书{i}"} for i in range(10)]
random.shuffle(books)


@app.get("/books")
def get_books(page: int, size: int, sort: bool):
    results = books[(page - 1) * size:page * size]
    if sort:
        results.sort(key=lambda x: x["id"])

    return results
```

上面我们定义了 `sort`，它的类型是布尔值，这样写的好处：

- 在API文档上有下拉选择 true/false
- 在URL上这个参数的值可以非常随意，下面这些值都可以被识别为Python的 `True` ，类似相反的值为 `False`

```
http://127.0.0.1:8000/books?page=1&size=5&sort=1
http://127.0.0.1:8000/books?page=1&size=5&sort=True
http://127.0.0.1:8000/books?page=1&size=5&sort=true
http://127.0.0.1:8000/books?page=1&size=5&sort=on
http://127.0.0.1:8000/books?page=1&size=5&sort=yes
```

2-6 查询参数有多个值

需求场景

比如，URL上会携带多个不同的查询参数，`/book?page=1&size=6&sort=on`，此时后端在路径函数中定义形参时有无特殊要求？

比如，URL上可能携带多个相同的查询参数，`/books?id=10&id=20`，此时如何获取多个id这个查询参数的多个值？

FastAPI的解决方式

示例1：在路径函数内定义多个查询参数时，需要遵循Python形参顺序的基本原则：先位置参数，后关键字参数

```python
# 正确的顺序
@app.get("/books")
def get_books(page: int, size: int = 4, sort: bool = False):
    results = books[(page - 1) * size:page * size]
    if sort:
        results.sort(key=lambda x: x["id"])
    return results
```

```
# 错误的顺序
@app.get("/books")
def get_books(page: int, size: int = 4, sort: bool):
    results = books[(page - 1) * size:page * size]
    if sort:
        results.sort(key=lambda x: x["id"])
    return results
```

示例2：获取URL中相同参数的多个值，需要使用Query()函数

```
import typing
from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/books")
def get_books(id: typing.List[int] = Query()):
    return id
```

2-7 查询参数使用Query校验

需求场景

就好比我们可以使用 `Path` 来给路径参数做更多的校验一样，我们可以使用 `Query` 来给查询参数做更多的校验

那使用Query的时候，该如何设置查询参数的默认值，如何设置参数必选，可选？

FastAPI的解决方式

示例1：使用Query()做字符串校验和数字的校验（使用方式和Path几乎一致）

```
from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items")
def info(name: str = Query(min_length=3), age: int = Query(gt=18)):
    return {"name": name, "age": age}
```

示例2：使用Query()设置查询参数的默认值

```python
from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items")
def info(name: str = Query(default="小小明", min_length=3)):
    return {"name": name}
```

示例3：使用Query()设置查询参数是可选的

```python
import typing

from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items")
def info(name: typing.Optional[str] = Query(default=None, min_length=3)):
    return {"name": name}
```

- 补充：参数是可选的只要设置default=None即可，`typing.Optional[str]`是为了做类型提示和自动补全

示例4：使用Query()设置查询参数是必选的，默认不使用 `default`即可

```python
import typing

from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items")
def info(name: str = Query(min_length=3)):
    return {"name": name}
```

补充：如果想显示的使用`default`设置参数是必选的，也可以使用特殊的语法：`default=...`

示例4：设置参数参数是必选的，但是值可以为None

```python
import typing

from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items")
def info(name: typing.Optional[str] = Query(default=..., min_length=3)):
    return {"name": name}
```

2-8 路径参数和查询参数的顺序问题

需求场景

你是否有疑问：一个接口中既有路径参数，又有查询参数，FastAPI是如何区分它们的，和顺序有关系吗？

你是否有疑问：一个接口中的路径参数和查询参数的顺序有关系吗，为什么我定义参数会有语法错误，无法运行程序？

FastAPI的解决方式

示例1：FastAPI知道如何区分：路径参数/查询参数，和顺序无关

```python
import typing

from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/{name}")
def info(name: str, age: str, id: typing.List[int] = Query(default=[1, 2])):
    return {"name": name, "age": age, "id": id}
```

原则：

- 路径函数内的形参名字不能重复
- 普通类型的参数默认会被识别为查询参数，比如：`int`，`float`，`str`，`bool`；复杂类型需要配合Query()显示声明为查询参数
- 只要形参的名字出现在路径装饰器中定义的参数，则肯定是路径参数，即使使用了Query()

示例2：路径参数和查询参数的顺序问题，**原则：Python函数形参的顺序，先位置参数后关键字参数**

```
@app.get("/books")
def get_books(page: int, size: int = 4, sort: bool = False):
    results = books[(page - 1) * size:page * size]
    if sort:
        results.sort(key=lambda x: x["id"])
    return results
```

2-9 阶段总结

- 结论1：路径指的就是路由或者说URL，分静态路由和动态路由
- 结论2：动态路由中的参数，就是路径参数；只要定义在路径函数的形参和路径装饰器中的参数同名，则该形参就会被是被为路径参数
- 结论3：**路径参数默认是必须的**，且参数类型默认是字符串，可以使用路径转化器或者类型提示的方式做类型转换
- 结论4：跟在URL `?` 后面的键值对参数称之为 查询参数，默认是字符串类型，可以使用类型提示的方式做类型转化
- 结论5：普通类型的形参（int/str/float/bool），FastAPI会默认它为查询参数；也可以使用Query()显示声明
- 结论6：使用Path()给路径参数做校验，使用Query()给查询参数做校验；做API文档的设置
- 结论7：查询参数和路过参数的定义顺序无关，FastAPI能都自动是被它们
- 结论8：定义在路径函数内形参的顺序，需要遵循：**没有默认值的放在前面，有默认值的放在后面（python的语法要求）**